

```

"""
EGESHA Smart Parking System (Kenya)
-----
Modules: Slot Manager | Entry |
Availability Display | Billing | Payment &
Barrier | Reports
Data structures: min-heap (free slots),
hash map (active vehicles), SQLite
(persistent store).
Run: pip install flask && python app.py
-> http://127.0.0.1:5000
"""

import heapq, math, re, sqlite3, threading,
random, string
from datetime import datetime
from flask import Flask, jsonify, request,
render_template

DB_FILE, TOTAL_SLOTS = "egesha.db", 40
app = Flask(__name__)
lock = threading.Lock()          # one
request mutates parking state at a time

# ---- In-memory data structures (rebuilt
from the DB on start-up) ----
free_heap = []    # MIN-HEAP of free slot
numbers: pop() = nearest free bay in O(log
n)
active = {}       # HASH MAP plate -> ticket
row: O(1) lookup of parked vehicles

```

```

SCHEMA = """
CREATE TABLE IF NOT EXISTS slots(slot_id
INTEGER PRIMARY KEY, zone TEXT, status TEXT
DEFAULT 'free');
CREATE TABLE IF NOT EXISTS tariffs(id
INTEGER PRIMARY KEY, max_minutes INTEGER,
fee INTEGER); -- NULL = no upper limit
CREATE TABLE IF NOT EXISTS
tickets(ticket_id INTEGER PRIMARY KEY
AUTOINCREMENT, plate TEXT NOT NULL,
    slot_id INTEGER REFERENCES
slots(slot_id), entry_time TEXT NOT NULL,
exit_time TEXT,
    duration_min INTEGER, fee INTEGER, paid
INTEGER DEFAULT 0);
CREATE TABLE IF NOT EXISTS
payments(payment_id INTEGER PRIMARY KEY
AUTOINCREMENT,
    ticket_id INTEGER REFERENCES
tickets(ticket_id), amount INTEGER, method
TEXT, reference TEXT, paid_at TEXT);
"""

def db():
    c = sqlite3.connect(DB_FILE);
    c.row_factory = sqlite3.Row; return c

def init():
    """Create tables, seed slots + tariffs
    once, then rebuild heap and hash map."""
    with db() as c:
        c.executescript(SCHEMA)
        if not c.execute("SELECT 1 FROM

```

```

slots").fetchone():
    c.executemany("INSERT INTO
slots(slot_id, zone) VALUES(?,?)",
                [(i, "A" if i <=
TOTAL_SLOTS // 2 else "B") for i in
range(1, TOTAL_SLOTS + 1)])
    if not c.execute("SELECT 1 FROM
tariffs").fetchone(): # editable later:
tariffs are data, not code
        c.executemany("INSERT INTO
tariffs(max_minutes, fee) VALUES(?,?)",
                [(30, 0), (120,
50), (240, 100), (360, 300), (None, 500)])
        for r in c.execute("SELECT slot_id
FROM slots WHERE status='free'"):
            heapq.heappush(free_heap,
r["slot_id"])
            for r in c.execute("SELECT * FROM
tickets WHERE exit_time IS NULL"):
                active[r["plate"]] = dict(r)

def tariffs():
    with db() as c:
        return [dict(r) for r in
c.execute("SELECT max_minutes, fee FROM
tariffs ORDER BY max_minutes IS NULL,
max_minutes")]

# ---- Billing module ----
def minutes_parked(entry_iso):
    """Whole minutes elapsed, rounded up
(30m 20s counts as 31 min)."""
    return math.ceil((datetime.now() -
datetime.fromisoformat(entry_iso)).total_se

```

```

conds() / 60)

def compute_fee(minutes):
    """Scan tiers in ascending order; first
    tier whose limit covers the duration
    wins."""
    for t in tariffs():
        if t["max_minutes"] is None or
        minutes <= t["max_minutes"]:
            return t["fee"]

def clean_plate(raw):
    plate = re.sub(r"\s+", "", (raw or
    "").upper())
    return plate if re.fullmatch(r"[A-Z0-9]
    {5,8}", plate) else None    # e.g. KDA123A

# ---- Payment & barrier module ----
def release(plate, minutes, fee,
method="FREE", ref=None):
    """Finish the visit: store payment,
    free the bay (push back on heap), open the
    barrier."""
    t, now = active.pop(plate),
    datetime.now().isoformat(timespec="seconds"
    )
    with db() as c:
        c.execute("UPDATE tickets SET
        exit_time=?, duration_min=?, fee=?, paid=1
        WHERE ticket_id=?",
                    (now, minutes, fee,
        t["ticket_id"]))
        c.execute("UPDATE slots SET
        status='free' WHERE slot_id=?",

```

```

(t["slot_id"],))
    if fee:
        c.execute("INSERT INTO
payments(ticket_id, amount, method,
reference, paid_at) VALUES(?,?,?,?,?)",
                (t["ticket_id"], fee,
method, ref, now))
        heapq.heappush(free_heap, t["slot_id"])
        return {"barrier": "open", "plate":
plate, "minutes": minutes, "fee": fee,
"paid": True}

def fail(msg, code=400):
    return jsonify(error=msg), code

# ---- Routes ----
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/slots")
def slots():
    """Availability display module: every
bay with its state, plus tariff table."""
    with db() as c:
        rows = [dict(r) for r in
c.execute("SELECT slot_id, zone FROM slots
ORDER BY slot_id")]
        taken = {t["slot_id"] for t in
active.values()}
        for r in rows:
            r["occupied"] = r["slot_id"] in
taken
        return jsonify(slots=rows,

```

```

free=len(free_heap), total=len(rows),
tariffs=tariffs())

@app.route("/api/entry", methods=["POST"])
def entry():
    plate = clean_plate((request.json or
    {}).get("plate"))
    if not plate: return fail("Enter a
    valid number plate, e.g. KDA 123A.")
    with lock:
        if plate in active: return fail(f"
        {plate} is already parked.", 409)
        if not free_heap: return
        fail("Parking is full. Please wait for a
        bay.", 409)
        slot, now =
        heapq.heappop(free_heap),
        datetime.now().isoformat(timespec="seconds"
        )

        with db() as c:
            cur = c.execute("INSERT INTO
            tickets(plate, slot_id, entry_time)
            VALUES(?,?,?)", (plate, slot, now))
            c.execute("UPDATE slots SET
            status='occupied' WHERE slot_id=?",
            (slot,))
            active[plate] = {"ticket_id":
            cur.lastrowid, "plate": plate, "slot_id":
            slot, "entry_time": now}
            return jsonify(plate=plate, slot=slot,
            entry=now, barrier="open")

@app.route("/api/bill", methods=["POST"])
def bill():

```

```

        """Exit step 1: find the vehicle,
        compute time and fee. Free visits leave at
        once."""
        plate = clean_plate((request.json or
        {}).get("plate"))
        with lock:
            t = active.get(plate)
            if not t: return fail("No parked
            vehicle with that plate.", 404)
            mins =
            minutes_parked(t["entry_time"]); fee =
            compute_fee(mins)
            if fee == 0: return
            jsonify(release(plate, mins, 0))
            return jsonify(plate=plate,
            slot=t["slot_id"], entry=t["entry_time"],
            minutes=mins, fee=fee, paid=False)

@app.route("/api/pay", methods=["POST"])
def pay():
    """Exit step 2: simulated M-Pesa
    payment. Barrier opens only after
    success."""
    data = request.json or {}
    plate, phone =
    clean_plate(data.get("plate")),
    re.sub(r"\D", "", data.get("phone", ""))
    if not re.fullmatch(r"(254|0)
    [17]\d{8}", phone): return fail("Enter a
    valid Safaricom number, e.g. 0712345678.")
    with lock:
        t = active.get(plate)
        if not t: return fail("No parked
        vehicle with that plate.", 404)

```

```

        mins =
minutes_parked(t["entry_time"])
        ref = "SIM" +
"".join(random.choices(string.ascii_uppercase + string.digits, k=7)) # replace with
Daraja STK push
        return jsonify(release(plate, mins,
compute_fee(mins), "M-PESA", ref),
reference=ref)

@app.route("/api/report")
def report():
    """Reports module: today's revenue and
the last 8 completed visits."""
    today =
datetime.now().date().isoformat()
    with db() as c:
        rev = c.execute("SELECT
COALESCE(SUM(amount),0) FROM payments WHERE
paid_at LIKE ?", (today + "%",)).fetchone()
[0]

        recent = [dict(r) for r in
c.execute("SELECT plate, slot_id,
duration_min, fee FROM tickets WHERE
exit_time IS NOT NULL ORDER BY ticket_id
DESC LIMIT 8")]
        return jsonify(revenue_today=rev,
recent=recent)

if __name__ == "__main__":
    init()
    app.run(debug=True)

```

